

SkyMaskCV（古典CV） と SkyMaskCNN（CNN） の比較

同じ機械・同じ画像・同じ指標で測ったもの。再現手順は末尾。
指標の現在地は [LOG.md](#)、採否の基準は [CRITERIA.md](#)。

結論

	SkyMaskCV	SkyMaskCNN
未知画像での精度（mIoU）	0.935	0.950
適合率（建物を空と誤る誤り）	0.972	0.978
再現率（空の取りこぼし）	0.960	0.970
1フレーム（取り込み込み・同一環境）	4.3～4.8ms	15.9～18.4ms
実機モバイル	7.5ms（iPhone SE3・実測）	未計測
配信サイズ	10KB （gzip・依存なし）	約3.2MB（wasm 2.9MB + ort 96KB + モデル 199KB）
初回フレームまで	即時	約5.1秒（デスクトップ・高速回線）

精度は CNN が上。ただし差は思ったより小さく、代償は大きい。
未知画像での mIoU 差は **+0.015**、適合率の差は **+0.006**。
その 0.015 のために、**速度4倍・配信量300倍・起動5秒**を払う。

30fps（33ms）の予算に対し、CV は実機で 7.5ms と4倍の余裕がある。
CNN の実機は未計測で、ここが最大の未確認事項。

この比較で最初に確認すべきだったこと

素朴に測ると CNN の圧勝に見えるが、その数字は使えない。理由が2つある。

1. 正解ラベルは CNN の教師の出力

評価に使っている labels_skyseg は SkySeg (U-2-Net) で作った擬似ラベル。

SkyMaskCNN の同梱モデルは、その SkySeg を教師にした蒸留モデル。

つまり CNN は「教師にどれだけ似ているか」で採点されており、構造的に有利。

教師が間違えている箇所では、CNN はその間違いを再現して加点され、

CV は正しくても減点されうる。どちらも人手の正解では測れていない。

2. データセットは CNN の学習データそのもの

CNN の学習データは Open Images V7 の 3,901 枚で、これは train スプリットと同一。

val 335 枚は CNN の検証セット。回帰セットに足した8枚も train/val 由来。

したがって**判断に使えるのは img/ 由来の19枚だけ**（手元の動画と静止画なので

Open Images に含まれず、CNN は一度も見えていない）。

区分	CV mIoU	CNN mIoU	差
img/ 由来 19枚 (CNN 未学習)	0.935	0.950	+0.015
データセット由来 8枚 (CNN 学習済み)	0.732	0.965	+0.233
val 335枚 (CNN の検証セット)	0.794	0.877	+0.083

見えていない画像では +0.015、学習した画像では +0.233。**この差が汚染の大きさ。**

train では測っていない (CNN の学習データなので、測る意味がない)。

精度

しきい値 0.5 で2値化して採点。CV の処理解像度のグリッドで比較する。

	CV mIoU	CV 適合率	CV 再現率	CNN mIoU	CNN 適合率	CNN 再現率
img/ 由来 19枚	0.935	0.972	0.960	0.950	0.978	0.970
val 335枚	0.794	0.922	0.846	0.877	0.918	0.948

val では適合率がほぼ互角（0.922 対 0.918）で、CNN の優位は**ほぼ全部が再現率**（0.846 → 0.948、+0.102）。このアプリは「建物が空になる誤り」を「空を取りこぼす誤り」より重く見ると決めてあるので、**重く見ている側の軸では差がついていない**。CNN は空を取りこぼさないのが強み。

CNN の弱点も CV と同じ向きに出る。既知の限界として「明るく平坦な壁面を空と誤判定する」と記載があり、これは CV の「霞んだ壁面・空を映すガラス」と同じ失敗。

しきい値の感度は小さい（val で 0.3～0.7 のあいだ mIoU 0.876～0.879）。上げると適合率が上がり再現率が下がる、素直な挙動。

速度

ヘッドレス Chrome（`--disable-gpu`）、1枚あたりの中央値。同一プロセスで交互に実行。

	CV 分離のみ	CV 取り込み込み	CNN 取り込み+推論+後処理
回帰セット	3.30ms	4.80ms	18.40ms
val	3.10ms	4.30ms	15.90ms

CNN の `segment()` は取り込み（512×512 の `getImageData`）を含むので、並べるなら CV も取り込み込みで測る。**約3.5～4倍**。

- CNN の初回推論は 54ms（wasm のウォームアップ）、2回目以降 17ms
- CNN は `ort.env.wasm.proxy = true` で Worker に出しているため、推論が遅い端末でもメインスレッドの描画は止まらない。**fps は落ちるが固まらない**
- wasm はシングルスレッド固定（GitHub Pages が COOP/COEP を付けられず `SharedArrayBuffer` が使えないため）。スレッドが使えれば縮む余地はある

実機モバイルは CV のみ実測（iPhone SE3 でフレーム全体 7.5ms）。
CNN は未計測。wasm はモバイルでデスクトップの3～5倍かかることが多いので
50～90ms/frame と見込まれるが、これは推測であって計測ではない。
30fps を要件にするなら、ここを測るまで CNN は採用可否を判断できない。

配信コストと起動

	SkyMaskCV	SkyMaskCNN
自前のコード	dist/skyMask.js 28KB（gzip 10KB）	sky-segmenter.js 12KB（gzip 4.6KB）
モデル	—	199KB
ランタイム	—	ort.min.js 96KB + wasm 約2.9MB （いずれも brotli 後）
合計	約10KB	約3.2MB
外部依存	なし	cdn.jsdelivr.net （落ちると動かない）
準備完了まで	即時	約5.1秒（デスクトップ・高速回線での実測）

初回フレームまでの5秒は、モバイル回線ではさらに伸びる。
CNN 側はその間 stats に「モデルを読み込んでいます…」と出す作りになっている。

それぞれが向く場面

CNN を採るべき場合

- 空の取りこぼしが問題になる（再現率が効く用途）
- 起動の数秒と3MBの通信が許される
- 実機で 33ms に収まることを確認できた

CV を採るべき場合

- 起動が速いこと、通信量が小さいこと、外部依存が無いことが効く
- 低速端末でも fps を落としたいくない

- 挙動が説明可能で、失敗の原因を追える必要がある

両方を採る手もある。 出力の性質が違う（CV は2値、CNN は確率）。

CNN が読み込めるまで CV で動かし、準備できたら差し替えれば、

起動の5秒を消したまま精度を取れる。両者は入力も表示も同じ作りなので差し替えは容易。

この比較で測っていないこと

- **実機モバイルでの CNN の速度**（最大の未確認事項）
- **人手の正解での精度**。どちらも教師の擬似ラベルでしか測っていない
- 動画のフレーム間の安定性。どちらも時間方向の平滑化を入れていない
- 消費電力・発熱。連続撮影では fps 以上に効くことがある

再現手順

```
# WebAppUtils（両リポジトリの親）を配信する
python3 tools/bench_server.py 8810 /tmp/bench.json
# 別のシェルから
open 'http://127.0.0.1:8810/SkyMaskCV/tools/bench.html?split=scenes'
```

tools/bench.html が両方を同じページで走らせ、結果を POST /result で書き出す。
入力はそれぞれが出荷している経路に揃えてある（CV は画素数一定に縮小した .bin、
CNN は元の JPEG から 512×512 で取り込み）。片方の前処理をもう片方に押し付けると、
設計判断ごと押し付けることになる。